

Relatório de Auditoria

Sistesma de Gestão de Salões — Saloes App

Data: 05 de Agosto de 2026

Versão: 0.0.0

Ambiente: Produção / Desenvolvimento

Sumário

1	Visão Geral e Resumo Executivo	
2	Arquitetura do Sistema	
3	Dependências Não Utilizadas	
4	Vulnerabilidades de Segurança	
5	Código Duplicado	
6	Melhorias de Performance	
7	Rotas Sem Autenticação	
8	Cobertura de Testes	
9	Fluxos Críticos do Negócio	
10	Plano de Ação Recomendado	

1. Visão Geral e Resumo Executivo

18

Dependências não utilizadas

4

Vulnerabilidades críticas

16+

Padrões de código duplicados

0%

Cobertura de testes

Área	Nota	Ações Prioritárias
Segurança	CRÍTICO	Remover bypass de service_role, fixar CORS, adicionar rate limiting, remover senhas em plaintext
Dependências	CRÍTICO	Remover 34+ pacotes não utilizados (~1.2MB economia no bundle)
Testes	CRÍTICO	0% cobertura — implementar testes mínimos para auth e pagamentos
Performance	MÉDIO	Code splitting, memoização, paginação, virtualização
Código Duplicado	MÉDIO	Refatorar 16+ padrões duplicados em componentes/hooks genéricos
Arquitetura	OK	Sólida — multi-tenant, RBAC, Supabase + Express estruturados

2. Arquitetura do Sistema

Stack Tecnológica

CAMADA	TECNOLOGIA	VERSÃO
Frontend	React + Vite + Tailwind CSS	React 18 / Vite 6
Roteamento	React Router DOM	6.26.0
Banco de Dados	Supabase (PostgreSQL)	2.108.2
Backend API	Express.js	5.2.1
Pagamentos	Asaas (gateway brasileiro)	—
UI Components	Radix UI + shadcn/ui	24+ pacotes
Gráficos	Recharts	2.15.4
Ícones	Lucide React	1.22.0

Estrutura de Diretórios

```
src/
├─ api/           # Camada de dados (Supabase client)
├─ components/    # Componentes reutilizáveis
│  ├─ schedule/  # Calendário, modais de agendamento
│  ├─ settings/  # Configurações da empresa
│  ├─ services/  # Modal de serviços
│  ├─ invoices/  # Modais de faturamento
│  ├─ plans/     # Seletor de modalidades
│  ├─ auth/      # Hook useCurrentUser
│  ├─ dashboard/# StatsCard
│  └─ companies/# Formulário de empresa
├─ ui/           # 40+ componentes shadcn/ui
├─ hooks/        # Hooks customizados
├─ lib/          # AuthContext, NavigationTracker
├─ pages/        # 20 páginas da aplicação
├─ types/        # Definições de tipo
└─ utils/        # Utilitários

server/
├─ index.js      # Pipeline de middleware + rotas
├─ lib/          # Supabase client, Asaas client
└─ routes/       # cep, asaas, whatsapp, email, auth
```

Fluxo de Dados

```
Frontend → Supabase (direto, anon key + RLS) → Leituras/Escritas
Frontend → Express Server (JWT do usuário) → Asaas, WhatsApp, E-mail, Auth Admin
Express  → Supabase (service_role key) → Operações admin (CRUD usuários)
```

Hierarquia de Roles

`super_admin` > `admin` > `profissional` > `cliente`

Multi-tenant: Empresas separadas por `company_id` / `company_ids` . Usuários podem pertencer a múltiplas empresas via tabela `customer_companies` .

3. Dependências Não Utilizadas

3.1 Dependências Confirmadas como Não Utilizadas (18 pacotes)

Nenhum `import` ou `require` encontrado em `src/` ou `server/` :

#	PACOTE	OBSERVAÇÃO
1	<code>three</code>	Biblioteca 3D (~600KB) — nenhum uso
2	<code>react-leaflet</code>	Nenhum componente de mapa
3	<code>react-quill</code>	Editor rich-text não utilizado
4	<code>react-markdown</code>	Nenhum render de markdown
5	<code>canvas-confetti</code>	Nenhum efeito de confetti
6	<code>html2canvas</code>	Nenhum screenshot
7	<code>jspdf</code>	Nenhum PDF gerado
8	<code>moment</code>	Projeto usa <code>date-fns</code> (31 imports em 14 arquivos)
9	<code>lodash</code>	Nenhum import encontrado
10	<code>@hello-pangea/dnd</code>	Nenhum drag-and-drop
11	<code>@radix-ui/react-toast</code>	Projeto usa <code>sonner</code>
12	<code>react-hot-toast</code>	Projeto usa <code>sonner</code> (30+ imports)
13	<code>framer-motion</code>	Nenhuma animação (~100KB)
14	<code>zod</code>	Nenhuma validação de schema
15	<code>@hookform/resolvers</code>	<code>react-hook-form</code> usado, mas não o resolver
16	<code>@stripe/react-stripe-js</code>	Stripe NÃO é utilizado
17	<code>@stripe/stripe-js</code>	Stripe NÃO é utilizado
18	<code>jsonwebtoken</code>	Auth usa Supabase Auth, não JWT manual

3.2 Wrappers shadcn/ui Órfãos (15 pacotes)

Arquivos wrapper existem em `src/components/ui/` mas **não são importados** por nenhum componente ou página:

PACOTE	ARQUIVO WRAPPER
@radix-ui/react-context-menu	ui/context-menu.jsx
@radix-ui/react-menubar	ui/menubar.jsx
@radix-ui/react-navigation-menu	ui/navigation-menu.jsx
@radix-ui/react-progress	ui/progress.jsx
@radix-ui/react-scroll-area	ui/scroll-area.jsx
@radix-ui/react-aspect-ratio	ui/aspect-ratio.jsx
@radix-ui/react-avatar	ui/avatar.jsx
@radix-ui/react-collapsible	ui/collapsible.jsx
@radix-ui/react-radio-group	ui/radio-group.jsx
@radix-ui/react-slider	ui/slider.jsx
@radix-ui/react-toggle	ui/toggle.jsx
@radix-ui/react-toggle-group	ui/toggle-group.jsx
react-resizable-panels	ui/resizable.jsx
input-otp	ui/input-otp.jsx
next-themes	ui/sonner.jsx

3.3 Impacto

- **Bundle JS:** ~1.2MB+ de código desnecessário (mesmo com tree-shaking)
- **node_modules:** Aumento significativo no tempo de instalação e build
- **Manutenção:** Pacotes desatualizados podem gerar alertas de segurança

4. Vulnerabilidades de Segurança

4.1 Crítico

#	VULNERABILIDADE	LOCALIZAÇÃO	DETALHES
C-1	Service Role Key como bypass de autenticação	<code>server/index.js:36</code>	O token <code>SUPABASE_SERVICE_ROLE_KEY</code> é aceito como Bearer token. Qualquer pessoa com a chave ignora toda autenticação e RBAC.
C-2	Credenciais em plaintext no disco	<code>.env.local</code> , <code>server/.env</code>	Service role key JWT em texto puro. Pode ter sido commitado ao version control.
C-3	CORS configurado para permitir todas as origens	<code>server/index.js:18</code>	<code>cors({ origin: true })</code> reflete a origem do request — qualquer site pode fazer chamadas autenticadas à API.
C-4	Senhas armazenadas em texto puro no banco	<code>server/routes/auth.js:150-155</code>	Campo <code>temp_password</code> armazena a senha em plaintext na tabela <code>users</code> . Se o banco for comprometido, todas as senhas temporárias ficam expostas.

4.2 Alto

#	VULNERABILIDADE	LOCALIZAÇÃO
H-1	Sem rate limiting em nenhum endpoint	<code>server/index.js</code> — todo o arquivo. Vulnerável a brute-force, enumeration e DoS.
H-2	Endpoint de confirmação de email sem verificação de role	<code>server/routes/auth.js:6</code> — qualquer usuário autenticado (incl. cliente) pode confirmar email de outros.
H-3	Política de senha fraca (mínimo 6 caracteres)	<code>ResetPassword.jsx:98</code> , <code>SetPassword.jsx:49</code>
H-4	Service role usado para todas as requisições autenticadas	<code>server/lib/supabase.js:33</code> — cliente Supabase com <code>service_role</code> para toda operação, não apenas admin.

4.3 Médio

#	VULNERABILIDADE	LOCALIZAÇÃO
M-1	Sem validação de input nos endpoints	<code>asaas.js</code> (valor/pagamento), <code>whatsapp.js</code> (telefone), <code>email.js</code> (formato)
M-2	console.log com dados sensíveis no frontend	<code>dbClient.js:516,540,572</code> — emails, senhas temporárias, IDs
M-3	Mensagens de erro expõem detalhes internos	<code>asaas.js:170</code> , <code>whatsapp.js:64</code> , <code>auth.js:30</code>
M-4	Sem headers de segurança (Helmet)	<code>server/index.js</code> — ausência de CSP, X-Frame-Options, etc.
M-5	CustomerPortal público sem proteção frontend	<code>/CustomerPortal</code> e <code>/Portalcliente</code> — login interno, mas sem middleware server-side
M-6	Sem limite de body no Express	<code>express.json()</code> sem <code>limit</code> configurado — vulnerável a DoS via payloads grandes

4.4 Dados Expostos

ARQUIVO	CONTEÚDO	RISCO
.env.local	VITE_SUPABASE_URL + VITE_SUPABASE_ANON_KEY	Anon key é pública por design, mas URL expõe infraestrutura
server/.env	SUPABASE_SERVICE_ROLE_KEY (JWT completo)	CRÍTICO — acesso irrestrito ao banco
server/.env.example	URL real do Supabase (não placeholder)	Expõe infraestrutura em repositórios

5. Código Duplicado

5.1 Crítico

5.1.1 NewAppointmentModal / NewLessonModal — 95% idênticos

ARQUIVO	LINHAS
src/components/schedule/NewAppointmentModal.jsx	457
src/components/schedule/NewLessonModal.jsx	439

Única diferença: campo `modality` vs `service_category`. Todo o resto (imports, lógica, JSX) é idêntico.

Refatoração: Extrair componente único `NewServiceModal` parametrizado pelo nome do campo.

5.1.2 Normalização de roles — copiado em 7 arquivos

```
rawRole === "teacher" ? "profissional" : rawRole === "user" ? "cliente" : rawRole
```

Arquivos: `Companies.jsx:57`, `Layout.jsx:125`, `Dashboard.jsx:47`, `Customers.jsx:98`, `Schedule.jsx:51`, `Invoices.jsx:89`, `Settings.jsx:95`

O hook `useCurrentUser()` já implementa `normalizeRole()` mas é ignorado.

5.1.3 Fetch de usuário atual — 9 arquivos

Padrão `db.auth.me()` + `useState` duplicado em: `Customers.jsx`, `Dashboard.jsx`, `Invoices.jsx`, `Companies.jsx`, `Settings.jsx`, `Schedule.jsx`, `CalendarSettings.jsx`, `Home.jsx`, `Layout.jsx`.

5.1.4 Normalização de `company_ids` — 16 ocorrências

```
currentUser?.company_ids?.length  
? currentUser.company_ids  
: (currentUser?.company_id ? [currentUser.company_id] : [])
```

O hook `useCurrentUser()` já computa `companyIds` mas é ignorado.

5.2 Alto

PADRÃO	OCORRÊNCIAS	REFATORAÇÃO
CRUD mutations boilerplate	6 páginas (Services, Plans, StylistLevels, PunchCards, Companies, Customers)	Hook <code>useCrudMutation(entityName, queryKey)</code>
Dialog de confirmação de exclusão	7+ locais (Services, Plans, StylistLevels, PunchCards, Companies, Customers, Invoices)	Componente <code><ConfirmDeleteDialog></code>
Loading skeleton / empty state	7+ páginas	Componentes <code><LoadingSkeleton></code> , <code><EmptyState></code>
Page header	7+ páginas	Componente <code><PageHeader></code>
Search/filter bar	6 páginas	Componente <code><SearchFilterBar></code>
DropDownMenu de ações	6+ páginas	Componente <code><ActionsMenu></code>
Form modal wrapper	5 páginas	Componente <code><FormModal></code>
Erro toast em mutations	27+ ocorrências	Utilitário <code>handleError</code> padronizado

6. Melhorias de Performance

6.1 P0 — Crítico

6.1.1 Zero Code Splitting

Todas as 20 páginas são importadas de forma síncrona em `pages.config.js`. Nenhum uso de `React.lazy()` ou `Suspense`.
Impacto: Bundle único de **~1.2MB** (`dist/assets/index-BM1-5AQE.js` = 1.206.884 bytes). Todo usuário baixa código de TODAS as páginas na primeira visita.
Fix: Converter imports em `pages.config.js` para `React.lazy()`.

6.1.1 Vite sem otimizações de build

`vite.config.js` não configura: `manualChunks` (separar vendors), `build.target` (navegadores modernos), `cssCodeSplit`.

6.1.3 18 dependências pesadas não utilizadas

Pacotes como `three` (~600KB), `moment` (~300KB), `framer-motion` (~100KB), `lodash` (~70KB) aumentam o bundle mesmo com tree-shaking.

6.2 P0 — React

6.2.1 Zero uso de React.memo

Nenhum dos 80+ componentes é otimizado com `React.memo`. Componentes re-renderizam desnecessariamente.

6.2.2 Computações custosas sem memoização no Dashboard

`Dashboard.jsx:120-177` — `companyStats` itera `companies` × `customers` × `invoices` a cada render (complexidade $O(n \times m \times k)$).

6.2.3 DayCalendar com complexidade $O(n^2 \times m)$

`DayCalendar.jsx:30-89` — `customers.find()` dentro de loops múltiplas vezes para o mesmo customer ID.

6.3 P1 — Alto

PROBLEMA	IMPACTO	LOCALIZAÇÃO
AuthContext causa re-render global	Toda mudança de estado re-renderiza a árvore inteira	<code>AuthContext.jsx</code>
Fetch duplicado de usuário por página	2+ requests extras por navegação	9 páginas
Sem paginação — fetch de TODOS os registros	Cargas lentas e alto consumo de memória	<code>Customers.jsx</code> , <code>Invoices.jsx</code> , <code>Dashboard.jsx</code>
Sem virtualização de listas	Centenas de DOM nodes para listas grandes	<code>Customers.jsx</code> , <code>Invoices.jsx</code>
Sem staleTime no TanStack Query	Dados considerados stale imediatamente	<code>query-client.js</code>

6.4 P2 — Médio

PROBLEMA	LOCALIZAÇÃO
CSS global com transição em todos os elementos (* { transition: ... })	index.scss:138-142
Inline styles em vez de Tailwind — objetos criados a cada render	Layout.jsx , Dashboard.jsx
Login carrega 4 imagens Unsplash sem loading="lazy"	Login.jsx:135
Invalidation de queries excessivamente ampla	Múltiplas páginas

7. Rotas Sem Autenticação

7.1 Frontend — Rotas Públicas

ROTA	COMPONENTE	RISCO
/login	Login.jsx	OK — intencional
/forgot-password	ForgotPassword.jsx	OK — intencional
/reset-password	ResetPassword.jsx	OK — intencional
/set-password	SetPassword.jsx	OK — intencional
/CustomerPortal	CustomerPortal.jsx	MÉDIO — portal público com login interno
/Portalcliente	CustomerPortal.jsx	MÉDIO — alias da rota acima

7.2 Backend — Rotas Públicas

MÉTODO	ROTA	RISCO
POST	/api/cep/	MÉDIO — proxy para viacep.com.br sem rate limiting

7.3 Backend — Rotas com Auth mas sem RBAC Adequado

MÉTODO	ROTA	PROBLEMA
POST	/api/auth/confirm-email	Qualquer usuário autenticado (incl. cliente) pode confirmar email de outros
GET	/api/health	Retorna email e role do usuário — information disclosure

7.4 Tabela Completa de Rotas Protegidas

ROTA	ROLES PERMITIDAS
/ , /Home , /Dashboard	super_admin, admin, profissional
/Schedule , /Clientes , /Invoices	super_admin, admin, profissional
/Services , /Plans , /PunchCards	super_admin, admin, profissional
/Templates , /WaitingList	super_admin, admin, profissional
/Settings , /Companies , /CalendarSettings	super_admin, admin
/StylistLevels	super_admin, admin
/AuditLogs	super_admin apenas
/CustomerDetail	super_admin, admin, profissional

8. Cobertura de Testes

ASPECTO	STATUS
Framework de testes	NENHUM — nenhum jest, vitest, cypress, playwright instalado
Arquivos de teste	ZERO — nenhum <code>.test.*</code> ou <code>.spec.*</code> no projeto
Configuração de testes	NENHUMA — sem <code>jest.config.*</code> , <code>vitest.config.*</code>
Script de teste	AUSENTE — <code>package.json</code> só tem <code>dev</code> , <code>build</code> , <code>lint</code> , <code>typecheck</code>
Cobertura estimada	0%
Ferramenta de qualidade	APENAS ESLint (<code>npm run lint</code>)

Fluxos que deveriam ter testes mínimos:

- 1. Autenticação (login, logout, recuperação de senha)
- 2. CRUD de agendamentos (criar, editar, cancelar, repetir)
- 3. Processamento de pagamentos (Asaas — criar cobrança, assinatura)
- 4. RBAC (verificação de permissões por role)
- 5. Gestão de clientes (criar, editar, associar a empresas)

9. Fluxos Críticos do Negócio

9.1 Autenticação

COMPONENTE	ARQUIVO	DESCRIÇÃO
Login	src/pages/Login.jsx	signInWithPassword + Google OAuth via Supabase Auth
Auth Context	src/lib/AuthContext.jsx	Gerencia sessão, listening de auth events, normalização de roles
RBAC Server	server/index.js:56-118	Middleware <code>attachUserRole</code> + verificação por role
Proteção Frontend	src/components/ProtectedRoute.jsx	Verificação de <code>allowedRoles</code> no roteamento

Problemas: Sem MFA/2FA, sem lockout após tentativas falhas, senha mínima de 6 caracteres.

9.2 Pagamentos (Asaas)

COMPONENTE	ARQUIVO	DESCRIÇÃO
Rota Server	server/routes/asaas.js	Cliente, cobranças, assinaturas, subcontas
Client Asaas	server/lib/asaas.js	Wrapper para API REST do Asaas com cobrança via PIX
Faturas	src/pages/Invoices.jsx	CRUD completo com sync Asaas, status, envio via WhatsApp
Integrações	src/api/dbClient.js:600-605	Mapeamento das funções de cliente e subcontas

Nota: Stripe está instalado mas NÃO é utilizado — deve ser removido.

Problema: Chaves de API armazenadas em `company_integrations` sem mecanismo de rotação.

9.3 Agenda/Agendamentos

COMPONENTE	ARQUIVO	DESCRIÇÃO
Página	src/pages/Schedule.jsx	Visualizações semanal/diária/mensal, criar/editar/repetir
Calendários	DayCalendar.jsx , WeeklyCalendar.jsx , MonthCalendar.jsx	3 visualizações de calendário
Modais	NewAppointmentModal.jsx , NewLessonModal.jsx	Criação (código 95% duplicado)
Sync	src/lib/calendarService.js	Google Calendar e Outlook
Portal	src/pages/CustomPortal.jsx	Visualização e confirmação pelo cliente

9.4 Faturamento

COMPONENTE	ARQUIVO
Página	src/pages/Invoices.jsx
Detalhe	src/components/invoices/InvoiceDetailModal.jsx
Criação	src/components/invoices/NewInvoiceModal.jsx

Fluxo: Criação → Status (pending/received/overdue/cancelled) → Envio via WhatsApp

9.5 Gestão de Clientes

COMPONENTE	ARQUIVO
CRUD	src/pages/Customers.jsx , CustomerDetail.jsx
Multi-tenant	Tabela customer_companies — clientes em múltiplas empresas
Planos/Créditos	Sistema de punch cards e créditos

10. Plano de Ação Recomendado

Semana 1 — Segurança (URGENTE)

1. Remover bypass de `service_role` key em `server/index.js:36`
2. Configurar CORS com origens específicas (produção e desenvolvimento)
3. Adicionar `express-rate-limit` nos endpoints de auth e pagamento
4. Remover armazenamento de senha em plaintext (`temp_password`)
5. Adicionar validação de input com `zod` (já instalado) nos endpoints
6. Adicionar `helmet` para headers de segurança
7. Remover `console.log` com dados sensíveis do frontend

Semana 2 — Limpeza de Dependências

1. Remover 18 dependências não utilizadas do `package.json`
2. Remover 15 wrappers `shadcn/ui` órfãos e seus imports
3. Mover `concurrently` para `devDependencies`
4. Executar `npm run build` e verificar tamanho do bundle

Semana 3 — Performance

1. Implementar `React.lazy()` em `pages.config.js` para code splitting por rota
2. Configurar `manualChunks` no `vite.config.js`
3. Adicionar `useMemo` nas computações custosas do Dashboard
4. Substituir fetch manual de usuário pelo hook `useCurrentUser()`
5. Adicionar paginação nos fetches de dados

Semana 4 — Refatoração

1. Mesclar `NewAppointmentModal` e `NewLessonModal` em componente único
2. Criar componentes reutilizáveis: `<PageHeader>`, `<ConfirmDeleteDialog>`, `<EmptyState>`, `<SearchFilterBar>`
3. Criar hook `useCrudMutation` para CRUD genérico
4. Padronizar erro handling com utilitário `handleError`

Semana 5-6 — Testes

1. Instalar Vitest + React Testing Library
2. Escrever testes para: autenticação, RBAC, CRUD de agendamentos, pagamentos
3. Configurar CI com `npm test` e `npm run lint`